# Supply Chain Pipeline

## 1. Overview

This document explains how data enters the pipeline (ingestion) and how Apache Flink processes it. The pipeline simulates a cold-chain supply network in Egypt: warehouses consume stock, trucks deliver replenishment orders, and the system watches truck telemetry for anomalies.

The main idea is a **closed loop**. Nothing is ordered by hand:

1. Sites consume stock (simulated demand).
2. Flink notices a shortage and creates an order automatically.
3. A Dispatcher service gives the order a free vehicle of the right cargo type.
4. The shipment goes back to the generator, a truck "drives" and sends telemetry.
5. The truck arrives, the stock goes up again, and the order is closed.

The flow of components is:

- **DataGenerator2.py** simulates the physical world and writes files into `data/`.
- **KafkaProducer.py** is the bridge: it sends those files to Kafka topics and returns dispatched shipments to the generator.
- **Kafka** (single broker) holds all topics.
- **Flink** runs four SQL jobs: Bronze, Silver, Detection and Replenishment.
- **Iceberg on MinIO** stores Bronze and Silver tables (Spark builds Gold later).
- **Dispatcher.py** turns orders into shipments.

## 2. Ingestion

Ingestion has two stages: the generator creates data as files, and the producer bridge moves those files into Kafka.

### 2.1 DataGenerator2.py: the simulated world

The generator does not make decisions. It only simulates what physically happens: stock goes down, trucks drive, devices send messy data.

**Master data (written once at start)**

| File | Content |
|---|---|
| products.csv | 4 products with a safe temperature range: Dairy (2 to 6 °C), Fresh Produce (4 to 10 °C), Frozen Food (-18 to -12 °C), Dry Goods (5 to 35 °C) |
| warehouses.csv | 28 warehouses: one per Egyptian governorate (27) plus Obour, with real coordinates |
| vehicles.csv | A fleet of 5,000 vehicles (`TRUCK-00001` and up). Types: refrigerated truck, reefer van, heavy truck. Capacities from 1,500 to 20,000 kg. |

| File | Content |
|---|---|
| | Existing vehicles are kept and only missing ones are added. |

The hub is `WH-11` (set by `START_WAREHOUSE_ID`). All trucks start at the hub, so every route is hub to destination. The other 27 warehouses are the "sites" that consume stock.

**Event files (continuously appended)**

| File | Kafka topic | Meaning |
|---|---|---|
| telemetry.jsonl | vehicle-telemetry | Truck readings (GPS, speed, engine, fridge temperature, weather) |
| inventory_stock.jsonl | inventory-stock | Stock level after every consumption or receipt |
| demand_events.jsonl | demand-events | Every consumption at a site |
| vehicle_status.jsonl | vehicle-status | A vehicle is available again after its turnaround |
| shipment_updates.jsonl | shipments | Shipment status = delivered |
| ground_truth.jsonl | not sent | Answer key for scoring the alert detector |

The generator also **reads** one file, `dispatch_inbox.jsonl`, which contains the shipments the Dispatcher created. It follows this file like a log (only new, complete lines), starting from the end of the file when it launches.

**Inventory and demand**

Each (site, product) pair, 108 in total, has its own stock record:

- A maximum level ("order up to") of 4,000, 6,000 or 8,000 kg, chosen randomly.
- Starting stock of 60% to 100% of that level.
- A reorder point at 35% of the maximum level.
- A lead time of about 260 seconds (trip duration, plus turnaround, plus a 30 s margin).
- Demand events of 50 to 250 kg, with random gaps (exponential distribution, mean 20 s, scaled by `DEMAND_SCALE`).

Every consumption writes two records: a demand event and a stock snapshot (`event_type = consumption`) one millisecond later. A delivery writes a snapshot with `event_type = receipt`. The first snapshot of each pair is `initial`. These event types matter later, because the replenishment job counts receipts.

**Trips and telemetry**

When a shipment arrives in the inbox, the generator starts a trip. The truck sends a reading about every 5 seconds (with 10% random variation), and the trip takes about 200 seconds. Route progress advances only while the truck is driving, so idling pauses the trip. Each reading contains around 40 fields: identifiers, GPS position, speed, RPM, fuel, OBD values, fridge temperature and limits, door state, shipment status, route progress and weather. Routes are straight lines by default; `--use-osrm` fetches

real driving routes, and `--use-weather` fetches historical weather from Open-Meteo. Both fall back to simulated values.

When progress reaches 100%, the last reading has `shipment_status = delivered`. The generator then adds the received quantity to the stock, writes the delivered shipment update, and frees the vehicle after 30 seconds.

**Anomaly episodes**

Anomalies last several readings, so that a duration-based detector can find them. A new anomaly starts with 1% probability per reading.

| Anomaly | What the generator does | Length |
|---|---|---|
| temperature_breach | Fridge temperature 2 to 6 °C above the product maximum | 35 to 65 s |
| excessive_idling | Speed 0, position frozen, engine on | 35 to 65 s |
| route_deviation | Position pushed 8 to 20 km sideways from the corridor line | 15 to 40 s |
| overspeed | Speed set to 140 km/h | one reading |

The generator also adds **harmless blips** (temperature or idling for 0 to 10 s, 1% probability). These are not anomalies, and a correct detector must ignore them. The `ground_truth.jsonl` file labels only the four real anomaly types, so `score_alerts.py` can compute precision and recall.

**Messy data**

The schema is never changed, but the way lines reach the file is made unreliable on purpose:

| Problem | How it is simulated |
|---|---|
| Timestamp jitter | Event time shifted by up to ±0.3 s |
| Duplicates | 2% of events are written twice, 70% of copies within 3 s and the rest 10 to 40 s later |
| Late events | 3% are delayed: 70% by 1 to 8 s (inside the watermark), 30% by 15 to 45 s (after the watermark) |
| Offline gaps | 0.4% chance per event that the device goes offline for 10 to 25 s. 70% of the gaps send their buffered events as a burst afterwards, and 30% lose them for good |

Technically this is done with an *outbox*: a priority queue of lines, each with a release time. Every loop iteration writes the lines whose time has come, so late lines and duplicates end up out of order in the file.

**Main loop**

Each pass of the loop (every 10 ms) does five things: (1) start trips for new shipments in the inbox, (2) generate demand at the sites, (3) create telemetry for every truck that is due, (4) handle arrivals (stock up, shipment delivered, vehicle released later), and (5) release due lines from the outbox. Files are flushed every 0.5 seconds.

### 2.2 KafkaProducer.py: the bridge to Kafka

The producer connects the files to Kafka. It runs three jobs at the same time.

**Job 1: create the topics.** At start it creates every missing topic with 3 partitions, including the topics that Flink and the Dispatcher will write to (alerts, replenishment-triggered, new-orders, shipments) and the dead-letter topic. This is why the README says to start the bridge before the Flink jobs.

**Job 2: send the data.**

- *CSV master data* goes to the `products`, `warehouses` and `vehicles` topics. Products and warehouses are sent once. Vehicles are re-read every 2 seconds and only changed rows are sent (detected with a SHA-256 fingerprint per row). Each CSV is read as a consistent snapshot, and the read is retried if the generator is writing at the same moment.
- *JSONL event files* are followed by a tail reader every 0.2 seconds. Each complete line is parsed and sent with a Kafka key.

| Topic | Kafka key |
|---|---|
| vehicle-telemetry | vehicle_id |
| inventory-stock, demand-events | site_id\|product_id |
| vehicle-status | vehicle_id |
| shipments | shipment_id |

Keys keep all events of one vehicle (or one site and product) in the same partition, so their order is preserved per key.

**Job 3: close the loop.** A Kafka consumer (group `generator-bridge`, starting from the latest offset) reads the `shipments` topic. Records with `status = dispatched` are copied into `dispatch_inbox.jsonl`, which the generator reads.

**Reliability features**

- Idempotent producer with `acks=all` and retries, so a retry cannot create duplicates inside Kafka.
- Bad lines (invalid JSON, not an object, or a missing `event_id` / `shipment_id`) are not lost: they go to the **dead-letter topic** `producer-dlq` with the error and the original value.
- If even the dead-letter topic fails, the record is written to a local file, `producer_failed_records.jsonl`, with an fsync. This file is not replayed automatically.
- Delivery callbacks count successes and failures, and a summary is printed at shutdown.

## 3. Flink Processing

### 3.1 Platform

Flink 1.20.5 runs in Docker with one JobManager and one TaskManager (8 task slots). A custom image adds the Iceberg runtime, the S3 client for MinIO, the Hadoop classes and the Kafka connector. Flink reads Kafka through the internal listener (`kafka:29092`) and writes Iceberg tables through a REST catalog, with data stored in the MinIO bucket `warehouse`.

Common settings in every job:

- Checkpoints every 30 seconds in exactly-once mode, with checkpoints retained if a job is cancelled.
- A fixed-delay restart strategy: up to 20 attempts, 10 seconds apart.
- A 10-second source idle timeout, so an empty partition does not block the watermark.
- Time zone UTC.

All four jobs are plain Flink SQL scripts started with the SQL client. They read from Kafka starting at the earliest offset, with separate consumer groups, so they work independently of each other.

### 3.2 Job 1: Bronze (01_bronze.sql)

Bronze is the raw, append-only copy of everything. A single Iceberg table, `lake.bronze.raw_events`, receives all 12 topics at once (selected with a topic pattern). Messages are read as plain text with no parsing, and each row records:

- Where it came from: topic, partition, offset, key and Kafka timestamp.
- The `payload`, exactly as received (JSON text).
- `ingest_ts`, the time Flink wrote it.
- `dt` and `hr`, the date and hour of the Kafka timestamp.

The table is partitioned by topic, date and hour. Duplicates and late events are kept on purpose, so the original data can always be replayed. The Iceberg sink commits at each checkpoint, which produces a new small file about every 30 seconds. Compaction is planned later (Airflow and Spark).

### 3.3 Job 2: Silver (02_silver.sql)

Silver is clean, typed data. One job writes three kinds of output from the telemetry and alerts topics.

**Telemetry processing, in this order:**

1. **Parse and check.** The JSON is parsed to typed columns, and `event_time_utc` is converted to a timestamp. A row goes to quarantine if it is unparseable (missing ids or time), has a bad speed (below 0 or above 250 km/h), or has bad GPS (outside the valid ranges).
2. **Deduplicate.** `ROW_NUMBER()` over `event_id` keeps the first copy.
3. **Late check.** An event older than the watermark (event time minus 10 seconds) is marked `late_event`. Deduplication happens first, so a late copy of an event that was already written counts as a duplicate, not as a late event.
4. **Write in event-time order.** Good rows go to `silver.telemetry`, partitioned by date and hour.

**Other outputs:**

- `silver.alerts`: alerts from the detection job, deduplicated by `alert_id`.
- `silver.quarantine`: every rejected row with a reason (`unparseable`, `bad_speed`, `bad_gps`, `late_event`) and the Kafka partition and offset, so the raw record can be found in Bronze.

Two design notes: the `ORDER BY event_ts` makes the telemetry insert run with parallelism 1, which is fine at this volume. Also, the column `temperature_breach_injected` is a label from the generator, and it should be dropped in Gold because a real sensor would not have it. Master and state topics (vehicles, shipments, orders) exist only in Bronze for now; Spark builds their Silver versions.

### 3.4 Job 3: Detection (03_detection.sql)

This job reads `vehicle-telemetry` directly and writes **one alert per anomaly episode** to the `alerts` topic.

| Rule | Condition |
|---|---|
| temperature_breach | Fridge outside the product's range for at least 30 s |
| excessive_idling | Engine on and speed below 1 km/h for at least 30 s |
| route_deviation | More than 5 km from the straight line hub to destination |
| overspeed | Speed above 120 km/h in a single reading |

**How episodes are found.** For the three duration-based rules, the job first selects the "bad" readings, then groups them per shipment with a **SESSION window with a 12-second gap**. Readings less than 12 seconds apart form one episode. With the 5-second send interval, one missing reading does not split an episode. The first three rules use `MIN` and `MAX` of event time to get the start, end and duration, and temperature and idling only alert when the duration is at least 30 seconds. This is why the generator's short blips (up to 10 s) are ignored, while real episodes (35 to 65 s) are caught.

**Route deviation** needs the position of the origin and destination. Instead of a join (which would add a second watermark to wait for), the job uses a constant `MAP` of the 28 warehouse coordinates. It converts degrees to kilometres with a flat-earth approximation around the origin, then computes the perpendicular distance from the GPS point to the corridor line. The generator pushes trucks 8 to 20 km away, well above the 5 km limit.

**Alert content.** Each alert has a deterministic `alert_id` (shipment, type and start time), the id of the first reading, the observed value and threshold, a readable detail text, and start time, end time and duration. A session is only closed when the watermark passes its end, so alerts appear a little after the episode finishes. Because the id is deterministic, a replay after a restart produces the same alerts, and Silver can deduplicate them.

The detection job reads the raw topic, not Silver. Duplicates do not hurt here, because `MIN` and `MAX` give the same result for repeated readings. Events later than the watermark are simply not used.

### 3.5 Job 4: Replenishment (04_replenishment.sql)

This job is the "brain" of the loop. It reads `inventory-stock` and `demand-events` and writes `replenishment-triggered` (why an order was created) and `new-orders` (the order itself). Both topics use a 5-second watermark.

**Step by step:**

1. **Merge the streams.** Stock rows and demand rows are combined into one stream per (site, product).
2. **Running state.** Two window calculations: the number of `receipt` events so far, and the demand (kg) in the last 60 seconds.
3. **Shortage check** on every stock row. A shortage is either `stock < reorder point` (reason `below_reorder_point`), or `stock / demand rate < lead time` (reason `projected_stockout`), where the demand rate is the last 60 s of consumption divided by 60.
4. **One open order at a time.** The order id contains the number of receipts seen so far: `ORD-<site>-<product>-<receipts>`. All triggers in the same open period have the same key, and only the first is kept (`ROW_NUMBER` ordered by processing time). When a delivery arrives, the receipt count increases, so a new shortage can create a new order. Because the id is always the same for the same shortage, a replay after a restart cannot create a different order, and the Dispatcher ignores ids it has already seen.
5. **Order quantity.** Refill up to the "order up to" level, but never less than half of that level and never more than 20,000 kg (the largest vehicle).
6. **Priority.** `URGENT` if stock is below half the reorder point, or the projected stock-out is within half the lead time. Otherwise `NORMAL`. The delivery deadline is two hours after the trigger.

The two outputs are written by one statement set, so they stay consistent.

**Why the Dispatcher is not Flink SQL.** Assigning a vehicle to an order uses up that vehicle, and two orders must never get the same truck. That needs shared, changing state, which streaming SQL cannot handle safely. Flink creates the orders, and `Dispatcher.py` allocates the fleet: an available vehicle with the right cargo type (refrigerated or dry), the smallest one that fits the quantity.

## 4. Kafka Topics

| Topic | Written by | Content |
|---|---|---|
| vehicle-telemetry | Producer bridge | Truck readings |
| inventory-stock | Producer bridge | Stock after each consumption or receipt |
| demand-events | Producer bridge | Every consumption at a site |
| vehicle-status | Producer bridge | Vehicle available again |
| vehicles, warehouses, products | Producer bridge | Master data |
| alerts | Detection job | One alert per anomaly episode |

| Topic | Written by | Content |
|---|---|---|
| replenishment-triggered | Replenishment job | Why an order was created |
| new-orders | Replenishment job | The automatic order |
| shipments | Dispatcher (dispatched), bridge (delivered) | Order, vehicle and trip |
| producer-dlq | Producer bridge | Records that could not be sent |

## 5. Running the Pipeline

1. Start the infrastructure: `docker compose -f Flink-Kafka-docker-compose.yml up -d --build`.
2. Start the generator: `python DataGenerator2.py`.
3. Start the bridge: `python KafkaProducer.py` (creates all topics).
4. Submit the four Flink jobs with `sql-client.sh -f` (01 to 04). Each one keeps running after the command ends.
5. Start the dispatcher: `python Dispatcher.py`.

With the default `DEMAND_SCALE=1`, the first order appears after about 5 to 6 minutes, because stock has to fall to the reorder point. Raising `DEMAND_SCALE` to 5 shows the full loop in about a minute. Progress can be checked in the Flink UI (port 8082: four running jobs, a checkpoint every 30 s), in the MinIO console (port 9001: Parquet files in `bronze/` and `silver/`), and with `score_alerts.py`, which compares alerts with `ground_truth.jsonl`.

## 6. Design Decisions and Things to Know

- **Bronze keeps everything.** Cleaning happens in Silver, and the raw record can always be traced through topic, partition and offset.
- **Messy data is added when events are written**, never in the schema, so the detection and cleaning logic is tested against realistic problems.
- **Event time with a 10-second watermark** is the line between "late but accepted" (1 to 8 s late) and "too late" (15 to 45 s late), which matches the generator's late-event settings.
- **Deterministic ids** (`alert_id`, `order_id`, shipment id from the order) make the pipeline safe to replay after a restart.
- **Restart behaviour of the bridge.** The producer tails each JSONL file from the beginning when it starts, so restarting it re-sends the existing file content to Kafka. The result is duplicate records, which Silver removes (for telemetry and alerts) but Bronze keeps. Deleting `data/` before a clean start avoids this.
- **Iceberg REST catalog state** lives inside its container. If only that container is restarted, run `docker compose down -v` and start again.
- **Parallelism.** The ordered telemetry insert in Silver runs with parallelism 1. If volume grows, remove the `ORDER BY`.